

LAB 5: Object-Oriented Programming & Exception Handling

- **Syllabus Alignment:** Unit 3: Object-Oriented Programming (10 LHs) — OOP Terminologies, Classes, Objects, Constructor/Destructor, Access Modifiers, Inheritance, Abstract Classes, Interfaces, Static elements, Namespaces, Exception Handling
- **Lab Objectives:** To apply encapsulation, manage structural data access properties, implement inheritance extension rules, define strict abstract rules, and manage runtime exception intercepts.
- **Lab Questions:**
 1. **Class Structuring & Object Interaction:** Design a library simulation engine using object-oriented principles. Define a `Book` class with attributes like *title*, *author*, *ISBN*, and *status*, along with a method to print its parameters. Next, define a `Member` class that can add or remove books from an internal collection tracking list.
 2. **Constructor & Destructor Lifecycles:** Build an asset inventory tracking class (`Pet`) that includes an initial structural `__construct` method to set your instance attributes, alongside an explicitly declared `__destruct` method that prints a confirmation message when an object instance is removed from system memory.
 3. **Encapsulation & Inheritance:** Create a base class named `Person` containing protected attributes for *name* and *email*, alongside public getters and setters. Extend this into two separate derived classes: `Student` (adding *studentID* and *course*) and `Staff` (adding *employeeID* and *department*). Fully demonstrate encapsulation across these classes.
 4. **Interfaces and Abstract Structures:** Implement a strict system architecture by declaring an interface named `IVehicle` containing definitions for `start()`, `stop()`, and `fuelStatus()`. Create an abstract class `Vehicle` that implements this interface and enforces a global constant value. Finally, derive concrete classes (`Car`, `Truck`) from the abstract layout.
 5. **Exception Interception Routing:** Create an arithmetic calculation engine class with a specific `divide()` method. Implement exception handling states (`try-catch` structures) that detect division-by-zero operations and throw a clean, user-friendly exception notice.